

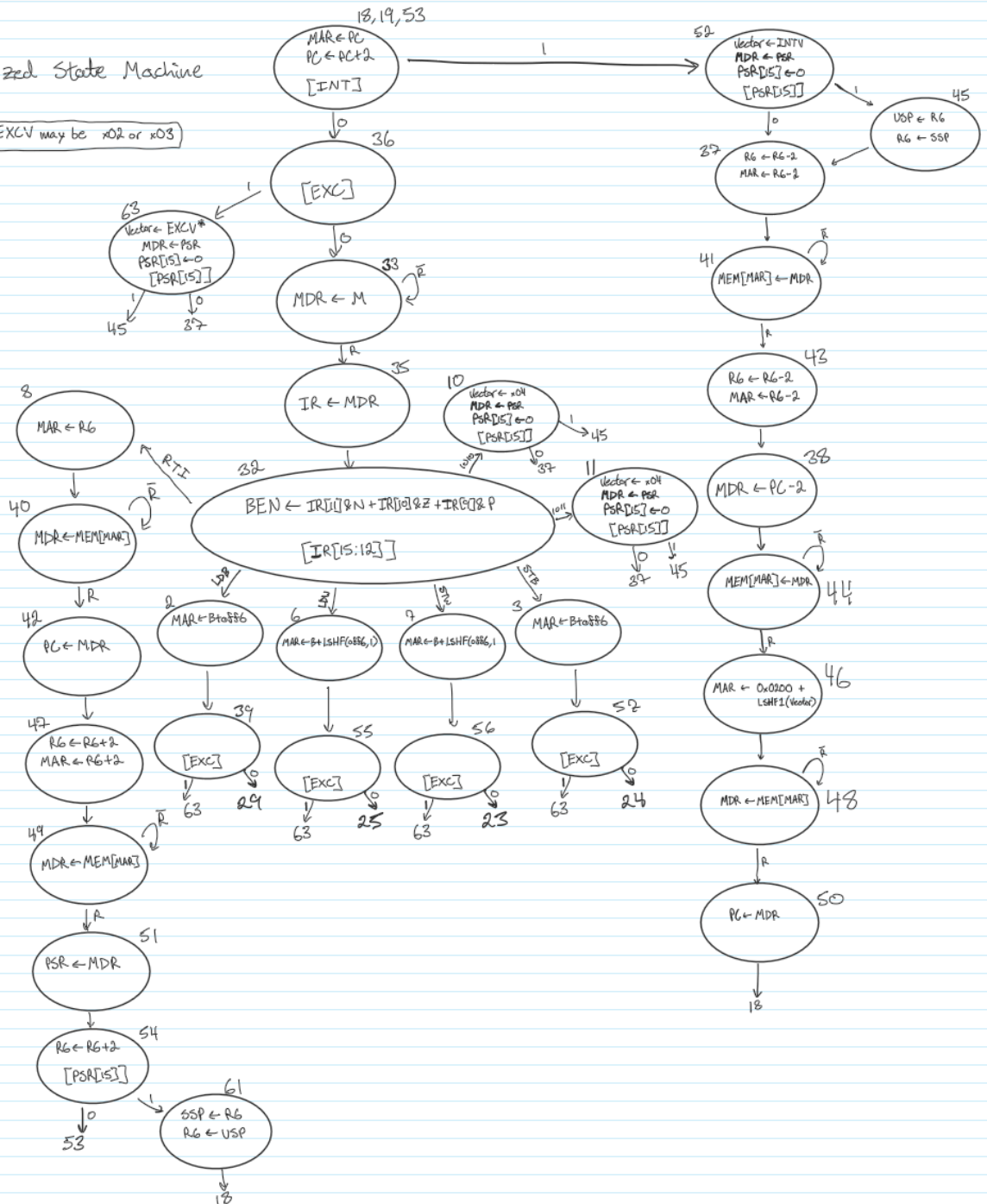
Lab 4 Documentation

Tyler Casey - TNC824

A1. State Machine

Finalized State Machine

Notes
* EXCV may be x02 or x03



Explanation:

There were new additions to the statemachine, I will explain them in sections.

Part 1: Interrupts

In the instruction fetch stage (18, 19, 53) I updated the state to include a check for the interrupt signal (updated microsequencer handles this). If that signal is present then we will branch to state 52 which loads INTV into vector and the processor is forced into supervisor mode $PSR[15] \leftarrow 0$, MDR gets the old PSR value and a check for what the old PSR[15] is occurs. If it is 1 we go to state 45 which will save USP and load SSP into R6, we will continue to state 37. If it is 0 we go to state 37 which begins the pushing of PSR and PC onto the supervisor stack; states (37, 41, 43, 38, 44) are responsible for this process. Finally state 46 calculates the handler address ($0x200 + LSHF1(\text{Vector})$) and loads it into PC states (48, 50). Then continues back to fetch state (18).

Part 2: Exceptions

(Addr Exceptions - Unaligned and Protection)

State 36 was added after fetch state (18, 19, 53) and checks if the EXC signal has been set, if so it will continue to state 63 which follows the same process as state 52, differing only in what value is loaded into the vector, (see notes on state machine) either x02 or x03 for protection or unaligned exceptions. The following sequence will be the same as interrupt. Additionally EXC check states are added after LDB, LDW, STW, and STB and are listed as states 39, 55, 56, and 57 respectively, if set they continue to state 63 if not they continue as normal to their next states 29, 25, 23, and 24.

(Unknown Opcode Exception)

States 10 and 11 will be the same as states 63 and 52, differing again in what value is loaded into the exception vector, in this case being x04 for unknown opcode exceptions.

Part 3: RTI

When the opcode is x1000, and has been decoded in state 32, we will continue to state 8 which will begin the process of popping the old PC and PSR values off the bus back into their registers, states (8, 40, 42, 47, 49, and 51) are responsible for this. A check is then had to see if the old PSR[15] value was 1 or 0 (Privilege or User) if so continue to state 61 which saves SSP and loads USP into R6, then continues back to state 18. If not we continue to fetch state 53 which had to be added to the fetch state for microsequencer logic.

Vector Generation:

Rather than using state machine logic to determine the exact exception vector, a VectorMUX was added. Selecting either INTV, x04, or the result of another MUX responsible for selecting either x02 or x03. This MUX takes the 1-bit PE and UE exception signals as a 2-bit select line ([PE, UE]). If PE is high (10), it selects vector x02. If UE is high (01), it selects vector x03, if both are high (11), priority says that protection exception should be handled first so x02 is set. A dedicated Vector register holds the output of the VectorMUX value, which is then left-shifted by 1 and gated onto the bus alongside the base address x0200 via the GateVector signal (top 8 bits are x02 and bottom 8 bits are the Vector).

A3. New Control Signals

LD Signals:

LD.Priv: Loads a new privilege bit (PSR[15]) into the Priv register.
 LD.SSP: Loads the current bus value into the Supervisor Stack Pointer (SSP) register.
 LD.USP: Loads the current bus value into the User Stack Pointer (USP) register.
 LD.Vector: Loads the chosen interrupt/exception vector into the Vector register.
 LD.EXC (PE / UE): Latches the Protection Exception (PE) or Unaligned Exception (UE) status so the exception vector can be generated.

Gate Signals:

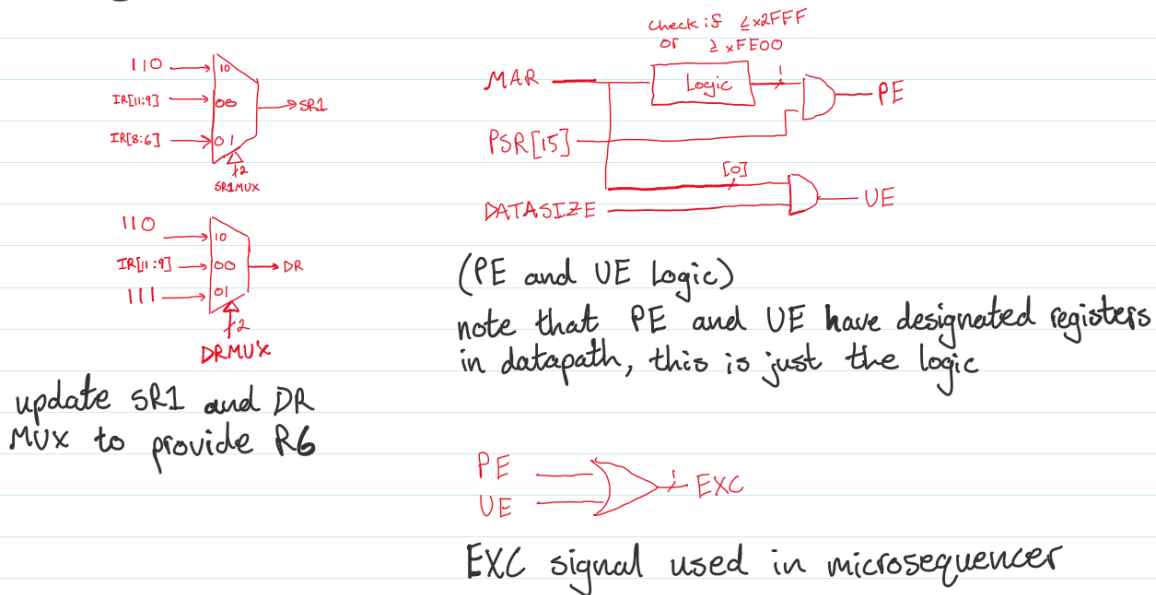
GatePSR: Gates the combined 16-bit Program Status Register (including the Priv bit and Condition Codes) onto the bus.
 GateSP: Gates the output of the SPMUX (one of the stack pointers or offset calculations) onto the bus.
 Gate Vector: Gates the left-shifted exception/interrupt Vector, concatenated with the base table address (x02), onto the bus.

MUXES:

PrivMUX (1 bit): Selects what gets loaded into the Priv register: either (1) a forced 0 (supervisor mode) or (0) the value from the bus (for restoring during RTI).
 PSRMUX (1 bit): Selects what updates the Condition Codes (N, Z, P): either (0) the standard ALU logic or (1) the value from the bus (for restoring during RTI).
 SPMUX (2 bits): Selects which stack pointer value to route to GateSP: (11) SSP, (10) USP, (01) R6-2 (for pushing), or (00) R6+2 (for popping).
 VectorMUX (2 bits): Selects which vector to load into the Vector register: (11) INTV, (10) x04 (Unknown Opcode), or (01) the datapath-calculated [PE, UE] vector (x02 or x03).
 Updated SR1MUX / DRMUX: An extra bit was added to the existing SR1MUX and DRMUX signals to allow an explicit selection of R6 (10) without relying on the Instruction Register.

A4. Additional logic/components

Control Structure



Explanation:

Updates to Existing Hardware:

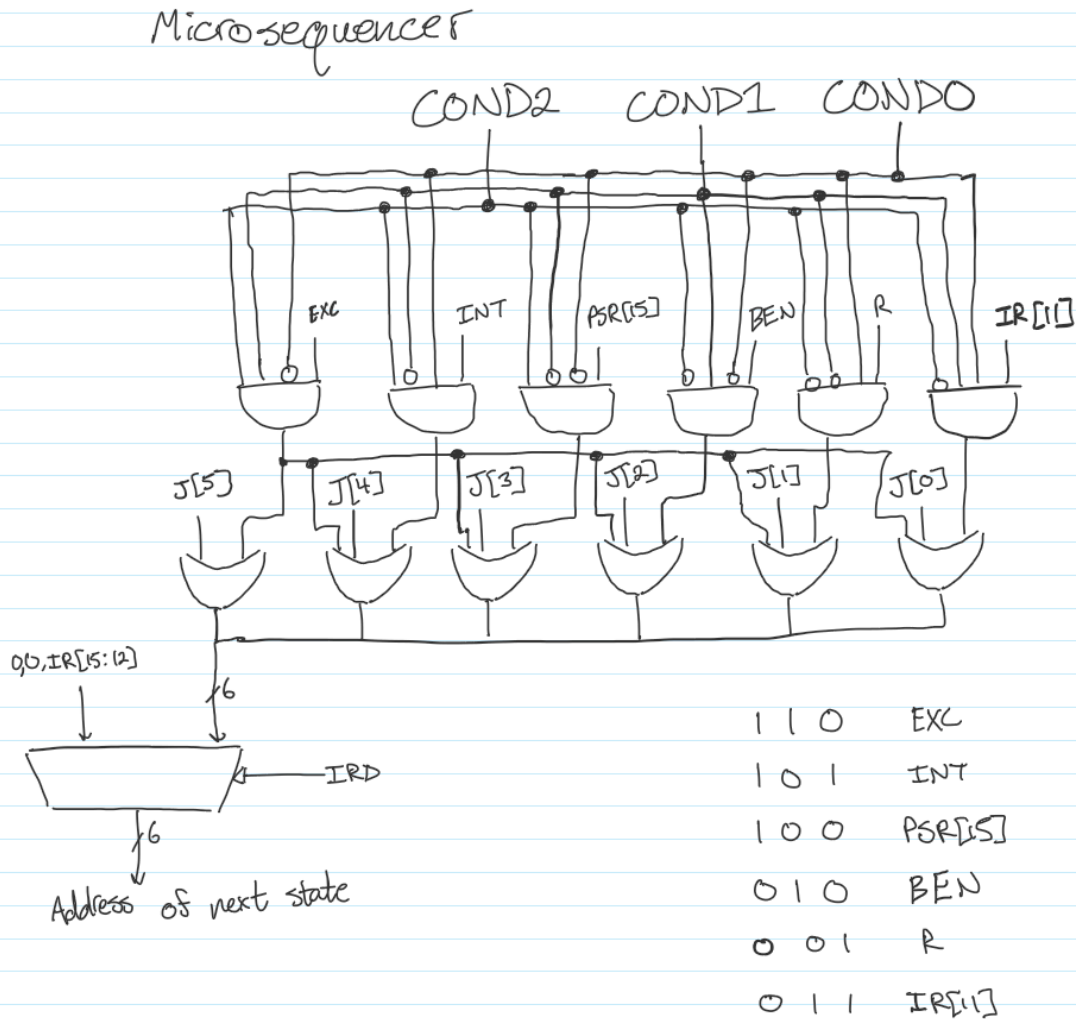
SR1MUX and DRMUX are updated to allow for another select bit, this allows for the addition of another input (R6) which is necessary for storing and restoring stack pointers without relying on the instruction register.

PE and UE Logic:

The Protection Exception (PE) is triggered via an AND gate if the MAR contains an address in system space ($< \text{x}3000$) AND the processor is currently in user mode ($\text{PSR}[15] = 1$). The Unaligned Exception (UE) is triggered via an AND gate if the lowest bit of the MAR is 1 (indicating an odd address) AND the DATA.SIZE signal indicates a word access.

These two signals are fed into an OR gate to generate a single EXC signal ($\text{EXC} = \text{PE} \mid \text{UE}$). This 1-bit EXC signal is sent to the microsequencer to trigger the exception microbranch, while the raw PE and UE signals remain in the datapath to drive the VectorMUX.

A5. Microsequencer



Explanation:

Shows the updated microsequencer used to access the additional states. Specifically there is an addition of another Cond bit to allow for the signals to be read. The signals added are EXC, INT, and PSR[15] (also known as the PRIV bit in the datapath).

EXC:

Is responsible for checking if there is an unaligned or protection exception (shown in additional logic components section), in which if the COND bits have been set to 110 the next state will be set to be 63. This was done as I wanted to check MEM exceptions (protection and unaligned) when MAR is updated, which happens in multiple different states and by allowing the signal to set all J bits high I can have the next state be the same as before (29, 25, 23, 24 for LDB, LDW, STW, STB).

INT:

Is the flag set by the processor when the timer interrupt is ready (set at cycle 300).

PSR[15]:

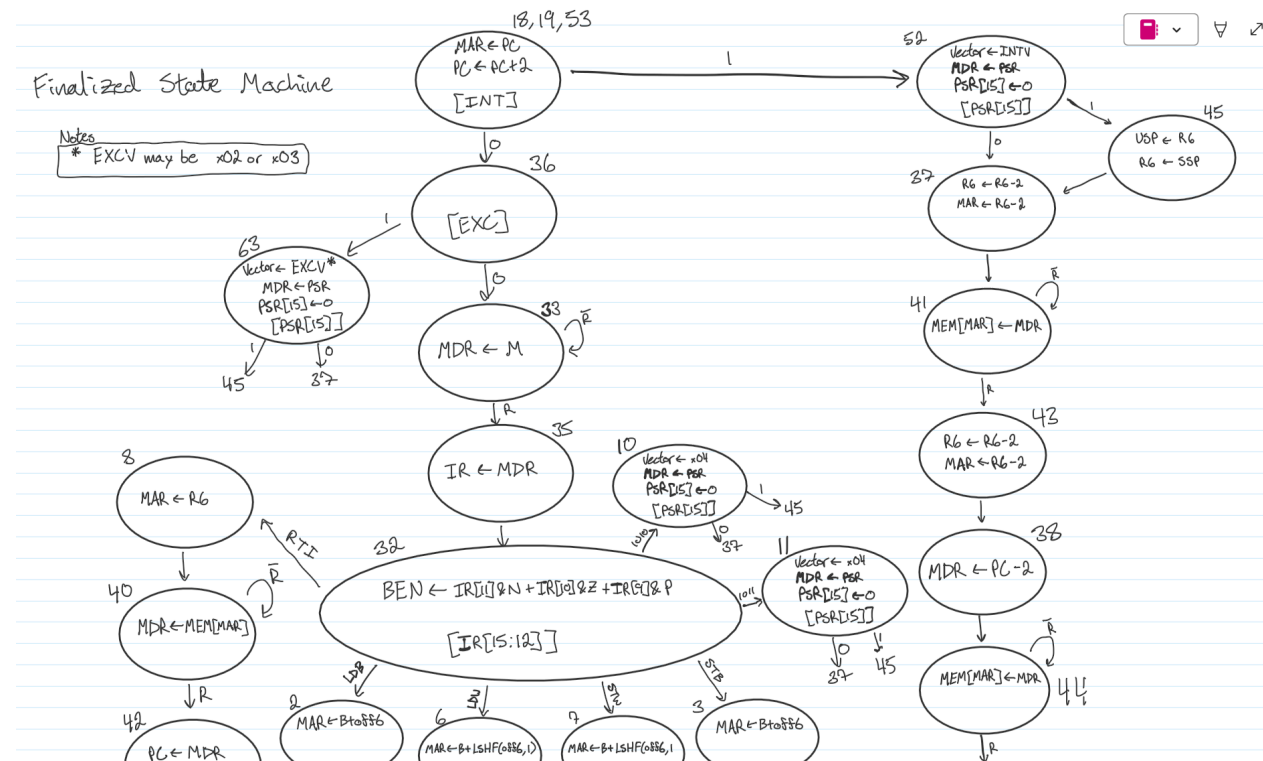
Shows whether user or supervisor mode, and will dictate whether the stack pointer (R6) needs to be swapped to the Supervisor Stack Pointer (SSP) during an interrupt/exception entry, or swapped back to the User Stack Pointer (USP) during an RTI instruction.

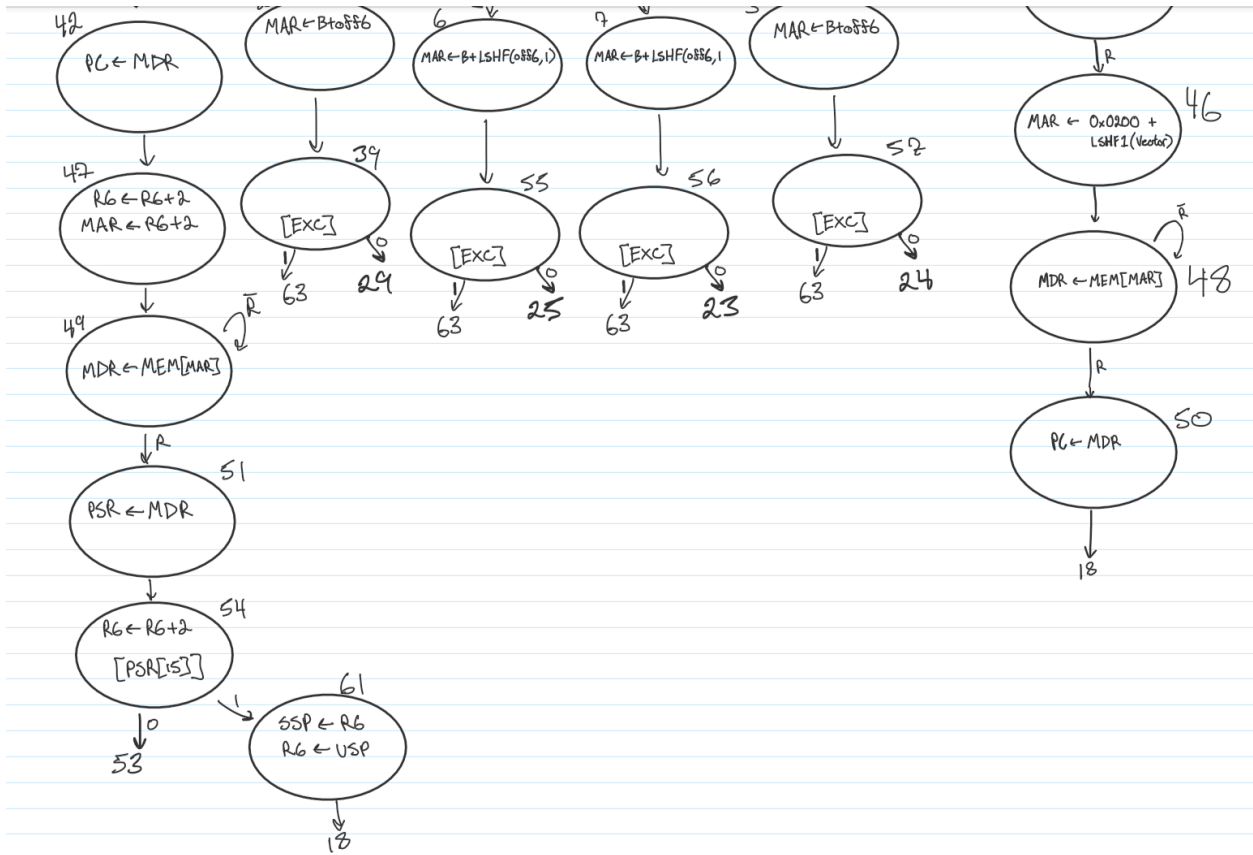
APPENDIX

Lists more close up photos in case the photos are hard to read/unfocused

Also actual file I used to work through the lab (notes and tests): Lab 4 .pdf

Statemachine:





Datapath:

